Checklist tÃ©cnico de implementaÃ§Ã£o local-first â€” patients, sessions e records

Objetivo
Transformar a direÃ§Ã£o arquitetural do Ethos em um checklist tÃ©cnico executÃ¡vel para os trÃªs primeiros domÃnios centrais da migraÃ§Ã£o local-first:
- `patients`
- `sessions`
- `records`

Este checklist foi pensado para uso direto por engenharia durante o refactor e a implementaÃ§Ã£o incremental.

Premissas
- IndexedDB serÃ¡ a base local oficial do Ethos Web.
- Repositories serÃ£o a camada de acesso ao dado.
- Domain services serÃ£o a camada de regra de negÃ³cio.
- O frontend deve migrar de API-first para local-first sem reescrita total.
- Sync remoto nÃ£o faz parte da primeira entrega desses domÃnios.

---

1. FundaÃ§Ã£o tÃ©cnica comum

1.1 Banco local
- [ ] criar o database adapter local
- [ ] definir nome do banco local
- [ ] definir versionamento de schema
- [ ] criar object stores para `patients`, `sessions` e `records`
- [ ] definir Ãndices mÃnimos por domÃnio
- [ ] implementar migraÃ§Ãµes locais iniciais

1.2 Contratos
- [ ] alinhar implementaÃ§Ã£o aos contratos de repositories
- [ ] alinhar implementaÃ§Ã£o aos contratos de domain services
- [ ] definir tipos compartilhados de erro de domÃnio
- [ ] definir polÃtica de timestamps (`createdAt`, `updatedAt`)
- [ ] definir polÃtica de `version`
- [ ] definir polÃtica inicial de `deletedAt`

1.3 Camada de infraestrutura
- [ ] criar pasta e estrutura dos repositories locais
- [ ] criar adapters locais por domÃnio
- [ ] garantir que a UI nÃ£o acesse IndexedDB diretamente
- [ ] garantir que hooks nÃ£o dependam de `fetch` direto para esses domÃnios

---

2. Patients

2.1 Schema local
- [ ] definir estrutura persistida de `Patient`
- [ ] definir campos obrigatÃ³rios
- [ ] definir campos opcionais
- [ ] definir Ãndice por `id`
- [ ] definir Ãndice para busca/listagem por nome ou status

2.2 Repository
- [ ] implementar `getById`
- [ ] implementar `list`
- [ ] implementar `create`
- [ ] implementar `update`
- [ ] implementar `archive`
- [ ] implementar `restore`
- [ ] implementar `remove`
- [ ] implementar `exists`

2.3 Domain service
- [ ] implementar `getPatient`
- [ ] implementar `listPatients`
- [ ] implementar `createPatient`
- [ ] implementar `updatePatient`
- [ ] implementar `archivePatient`
- [ ] implementar `restorePatient`
- [ ] implementar `deletePatient`

2.4 Regras de domÃnio
- [ ] validar criaÃ§Ã£o sem nome invÃ¡lido
- [ ] normalizar dados bÃ¡sicos de paciente
- [ ] impedir inconsistÃªncias Ã³bvias de update
- [ ] garantir que arquivamento nÃ£o destrua histÃ³rico
- [ ] definir polÃtica de exclusÃ£o destrutiva versus remoÃ§Ã£o lÃ³gica

2.5 IntegraÃ§Ã£o com frontend
- [ ] criar hooks `usePatients` e `usePatient`
- [ ] adaptar `PatientsPage` para consumir domain service
- [ ] adaptar `PatientDetailPage` para consumir domain service
- [ ] remover dependÃªncia direta de API remota nessas telas

2.6 Testes
- [ ] testar criaÃ§Ã£o local de paciente
- [ ] testar atualizaÃ§Ã£o local de paciente
- [ ] testar listagem filtrada
- [ ] testar arquivamento e restauraÃ§Ã£o
- [ ] testar recuperaÃ§Ã£o apÃ³s reload da aplicaÃ§Ã£o

---

3. Sessions

3.1 Schema local
- [ ] definir estrutura persistida de `Session`
- [ ] definir vÃnculo obrigatÃ³rio com `patientId`
- [ ] definir Ãndice por `id`
- [ ] definir Ãndice por `patientId`
- [ ] definir Ãndices por status e data, se necessÃ¡rio

3.2 Repository
- [ ] implementar `getById`
- [ ] implementar `list`
- [ ] implementar `listByPatient`
- [ ] implementar `create`
- [ ] implementar `update`
- [ ] implementar `remove`
- [ ] implementar `exists`

3.3 Domain service
- [ ] implementar `getSession`
- [ ] implementar `listSessions`
- [ ] implementar `listSessionsByPatient`
- [ ] implementar `createSession`
- [ ] implementar `updateSession`
- [ ] implementar `completeSession`
- [ ] implementar `cancelSession`
- [ ] implementar `markSessionNoShow`
- [ ] implementar `deleteSession`

3.4 Regras de domÃnio
- [ ] validar existÃªncia de `patientId` na criaÃ§Ã£o
- [ ] validar transiÃ§Ãµes de status
- [ ] impedir inconsistÃªncia temporal grave
- [ ] preservar integridade entre paciente e sessÃ£o
- [ ] definir polÃtica de remoÃ§Ã£o lÃ³gica ou destrutiva

3.5 IntegraÃ§Ã£o com frontend
- [ ] criar hooks `useSessions` e `useSession`
- [ ] adaptar `HomePage` no que depender de sessÃµes
- [ ] adaptar `SessionPage`
- [ ] adaptar navegaÃ§Ã£o baseada em sessÃ£o selecionada
- [ ] remover dependÃªncia direta de API remota nessas jornadas

3.6 Testes
- [ ] testar criaÃ§Ã£o local de sessÃ£o
- [ ] testar vÃnculo com paciente
- [ ] testar listagem por paciente
- [ ] testar mudanÃ§a de status
- [ ] testar persistÃªncia apÃ³s reload

---

4. Records

4.1 Schema local
- [ ] definir estrutura persistida de `ClinicalRecord`
- [ ] definir vÃnculo obrigatÃ³rio com `patientId`
- [ ] definir vÃnculo opcional com `sessionId`
- [ ] definir Ãndice por `id`
- [ ] definir Ãndice por `patientId`
- [ ] definir Ãndice por `sessionId`
- [ ] definir Ãndice por `kind`, se necessÃ¡rio

4.2 Repository
- [ ] implementar `getById`
- [ ] implementar `list`
- [ ] implementar `listByPatient`
- [ ] implementar `listBySession`
- [ ] implementar `create`
- [ ] implementar `update`
- [ ] implementar `remove`
- [ ] implementar `exists`

4.3 Domain service
- [ ] implementar `getRecord`
- [ ] implementar `listRecords`
- [ ] implementar `listRecordsByPatient`
- [ ] implementar `listRecordsBySession`
- [ ] implementar `createRecord`
- [ ] implementar `updateRecord`
- [ ] implementar `createSessionRecordDraft`
- [ ] implementar `deleteRecord`

4.4 Regras de domÃnio
- [ ] validar vÃnculo com paciente
- [ ] validar vÃnculo com sessÃ£o quando fornecido
- [ ] atualizar `version` a cada ediÃ§Ã£o
- [ ] atualizar `updatedAt` corretamente
- [ ] garantir compatibilidade com histÃ³rico clÃnico
- [ ] definir polÃtica conservadora de exclusÃ£o

4.5 IntegraÃ§Ã£o com frontend
- [ ] criar hooks `useRecords`, `useRecord`, `useRecordsByPatient`, `useRecordsBySession`
- [ ] adaptar `ProntuarioPage`
- [ ] adaptar fluxos que abrem prontuÃ¡rio a partir de sessÃ£o
- [ ] garantir que a UI nÃ£o precise montar sozinha o draft clÃnico

4.6 Testes
- [ ] testar criaÃ§Ã£o local de record
- [ ] testar listagem por paciente
- [ ] testar listagem por sessÃ£o
- [ ] testar update com versionamento
- [ ] testar criaÃ§Ã£o de draft a partir de sessÃ£o
- [ ] testar persistÃªncia apÃ³s reload

---

5. IntegraÃ§Ã£o entre domÃnios

5.1 Regras cruzadas
- [ ] garantir que sessÃ£o nÃ£o possa ser criada para paciente inexistente
- [ ] garantir que record nÃ£o possa ser criado para paciente inexistente
- [ ] garantir que record com `sessionId` aponte para sessÃ£o vÃ¡lida
- [ ] definir comportamento ao arquivar paciente com sessÃµes e records existentes

5.2 Fluxos compostos
- [ ] implementar fluxo â€œcriar sessÃ£o + preparar draft de prontuÃ¡rioâ€
- [ ] implementar fluxo â€œabrir paciente + listar sessÃµes e records locaisâ€
- [ ] implementar fluxo â€œconcluir sessÃ£o + garantir consistÃªncia do recordâ€

---

6. MigraÃ§Ã£o do frontend

6.1 Hooks

- [ ] mapear hooks atuais dependentes de API remota
- [ ] criar hooks novos orientados a domain services
- [ ] substituir consumo de API por consumo de domain services

6.2 PÃ¡ginas prioritÃ¡rias
- [ ] `PatientsPage`
- [ ] `PatientDetailPage`
- [ ] `SessionPage`
- [ ] `ProntuarioPage`
- [ ] `HomePage` no trecho dependente dos domÃnios centrais

6.3 NavegaÃ§Ã£o
- [ ] revisar seleÃ§Ã£o de paciente e sessÃ£o para nÃ£o depender de payload remoto
- [ ] manter navegaÃ§Ã£o funcional apÃ³s reload local

---

7. Remoto e compatibilidade temporÃ¡ria

7.1 Adapters remotos temporÃ¡rios
- [ ] manter adapters remotos separados por domÃnio
- [ ] evitar uso direto de `apiClient` em pÃ¡ginas
- [ ] documentar claramente o que ainda depende da API remota nesta fase

7.2 EstratÃ©gia de transiÃ§Ã£o
- [ ] definir quais telas entram primeiro em local-first
- [ ] definir fallback temporÃ¡rio enquanto a migraÃ§Ã£o Ã© parcial
- [ ] evitar mistura desorganizada entre storage local e API remota

---

8. SeguranÃ§a e privacidade

8.1 Dados locais
- [ ] validar que dados clÃnicos nÃ£o ficam expostos em texto puro fora da camada prevista
- [ ] revisar logs locais para evitar vazamento de conteÃºdo clÃnico
- [ ] preparar compatibilidade com camada de criptografia local

8.2 Restore e backup futuro
- [ ] garantir que schema e contratos permitam export futuro
- [ ] garantir que entities contenham metadados Ãºteis para backup e restore

---

9. CritÃ©rios de pronto da primeira entrega

A primeira entrega pode ser considerada bem-sucedida quando:
- [ ] patients funciona localmente como fonte primÃ¡ria
- [ ] sessions funciona localmente como fonte primÃ¡ria
- [ ] records funciona localmente como fonte primÃ¡ria
- [ ] pÃ¡ginas centrais consomem domain services em vez de API direta
- [ ] reload da aplicaÃ§Ã£o preserva dados locais
- [ ] o fluxo principal clÃnico deixa de depender do backend remoto nesses trÃªs domÃnios

---

10. Ordem recomendada de execuÃ§Ã£o

Ordem prÃ¡tica
1. fundaÃ§Ã£o do banco local
2. patients repository + service + hooks + pÃ¡ginas
3. sessions repository + service + hooks + pÃ¡ginas
4. records repository + service + hooks + pÃ¡ginas
5. fluxos compostos entre os trÃªs domÃnios
6. limpeza de dependÃªncias remotas diretas
7. testes de persistÃªncia e integridade

---

11. PrÃ³ximos artefatos recomendados

Depois deste checklist, os prÃ³ximos documentos mais Ãºteis sÃ£o:
1. schema inicial do banco local para patients, sessions e records
2. checklist tÃ©cnico da quebra do `Index.tsx`

3. plano de migraÃ§Ã£o dos hooks e pÃ¡ginas prioritÃ¡rias
4. matriz de dependÃªncias remotas que ainda restarem apÃ³s a primeira fase